A Real-Time Wildfire Resilience Digital Twin Using Streaming Geospatial Analytics

PRINCE SAVSAVIYA, University of California, Riverside, USA
VISWANADH R. CHALLA, University of California, Riverside, USA
ANISH ASHOK KALE, University of California, Riverside, USA
ALI REZAYI NEJAD, University of California, Riverside, USA

Wildfires pose a significant threat to infrastructure, ecosystems, and human safety, particularly in regions with dense urban development and climate-driven risk factors. Existing monitoring systems often lack real-time integration of geospatial data, infrastructure context, and interactive decision-making capabilities. This paper presents a real-time wildfire resilience digital twin that integrates streaming data ingestion, geospatial analytics, and interactive visualization to support rapid situational awareness and scenario analysis. The system leverages an event-driven architecture using Apache Kafka for data ingestion, Apache Spark for real-time processing, and DuckDB for low-latency analytical storage. Spatial modeling is achieved through hierarchical hexagonal indexing (H3) combined with large-scale building footprint data, enabling multi-resolution analysis from statewide overview to city-level detail. An interactive dashboard built with Streamlit and PyDeck enables users to visualize live alerts and simulate fire events under varying environmental conditions. The proposed system demonstrates scalable, low-latency performance while enabling intuitive exploration of wildfire risk, highlighting the potential of digital twin frameworks for disaster resilience and decision support.

Additional Key Words and Phrases: wildfire digital twin, geospatial analytics, Apache Kafka, Apache Spark, Apache Sedona, H3 indexing, Streamlit, PyDeck, real-time monitoring

Group Name: N-Dimensional
Student IDs: 862548420, 862548873, 862547289, 862392323

1 Introduction

Wildfires have become increasingly frequent and severe due to climate change, urban expansion, and environmental degradation, posing substantial risks to human life, infrastructure, and ecological systems. In regions such as California, the ability to monitor and respond to wildfire threats in real time is critical for effective disaster management and resilience planning. Traditional wildfire monitoring systems often rely on static data sources or delayed reporting mechanisms, limiting their effectiveness in rapidly evolving scenarios.

Recent advances in streaming systems, geospatial analytics, and interactive visualization have enabled the development of digital twin frameworks that mirror real-world environments in real time. A digital twin for wildfire monitoring can integrate heterogeneous data sources, including satellite-based fire detections, weather conditions, and infrastructure information, to provide a unified and continuously updated representation of risk.

This paper presents a real-time wildfire resilience digital twin system that combines event-driven data pipelines, scalable spatial modeling, and interactive visualization to support both monitoring and what-if analysis. The system ingests live and simulated wildfire events using Apache Kafka, processes them through a streaming pipeline, and integrates them with large-scale geospatial datasets using hierarchical indexing. A web-based dashboard enables users

Authors' Contact Information: Prince Savsaviya, psavs001@ucr.edu, University of California, Riverside, Department of Computer Science, Riverside, California, USA; Viswanadh R. Challa, vchal008@ucr.edu, University of California, Riverside, Department of Computer Science, Riverside, California, USA; Anish Ashok Kale, akale014@ucr.edu, University of California, Riverside, Department of Computer Science, Riverside, California, USA; Ali Rezayi Nejad, areza018@ucr.edu, University of California, Riverside, Department of Computer Science, Riverside, California, USA.

to explore risk patterns, visualize affected infrastructure, and simulate fire scenarios under varying environmental conditions.

The primary contributions of this work include: (1) a scalable, event-driven architecture for real-time wildfire data integration; (2) a multi-resolution geospatial modeling approach using H3 indexing and building footprint data; and (3) an interactive digital twin interface that supports simulation-driven decision-making. The remainder of the paper is organized as follows: Section 2 reviews related work, Section 3 describes the system architecture, Section 4 details the methodology, Section 5 discusses implementation, Section 6 presents evaluation results, and Section 7 concludes with future directions.

2 Related Work

Recent work on wildfire intelligence can be grouped into 3 broad categories: wildfire-specific digital twins, data-driven wildfire prediction and monitoring, and systems for geospatial sensemaking and streaming analytics. This project is most closely related to the first category. Recent reviews argue that wildfire digital twins require continuous integration of real-time sensing, simulation, and decision-support interfaces, and that the main open challenge is fusing heterogeneous dynamic feeds with operational geospatial context at usable latency. Our system follows this direction, but emphasizes an implementable streaming architecture that connects event ingestion, infrastructure-aware spatial querying, and interactive visualization in a single end-to-end pipeline.

A second line of post-2020 work focuses on wildfire prediction and monitoring from remote sensing and statistical or machine learning models. For example, Huot et al. introduced the *Next Day Wildfire Spread* dataset for forecasting wildfire spread from multivariate remote-sensing inputs, while Koh et al. modeled wildfire occurrence and size using spatiotemporal point processes with environmental covariates. These works are important because they improve predictive understanding of wildfire dynamics, but their primary contribution is predictive modeling rather than real-time systems integration. In contrast, the present project is centered on operational data fusion: ingesting events, contextualizing them with static infrastructure, and surfacing risk through a live digital twin interface.

A third category studies interactive wildfire visualization and broader decision-support tooling. These works motivate the interactive component of our system. However, our project differs in that the visualization layer is explicitly coupled to a streaming backend, a live alert store, and multi-resolution infrastructure context derived from large-scale building footprints.

Finally, several recent studies have moved closer to digital-twin-style wildfire systems. Halem et al. and Ahmed et al. describe dynamic and data-driven wildfire digital twin concepts, and Kwon discusses digital-twin-based prediction for wildland–urban interface fires. These efforts highlight the growing interest in wildfire digital twins as operational cyberinfrastructure. Relative to this literature, the present work contributes a course-scale but concrete implementation that combines Kafka-based event streaming, Spark/Sedona-oriented spatial reasoning, H3-based multi-resolution indexing, DuckDB-backed alert serving, and Streamlit/PyDeck visualization. Rather than focusing purely on forecasting accuracy or conceptual digital twin frameworks, the project emphasizes practical end-to-end integration of streaming and geospatial analytics for infrastructure-aware wildfire resilience.

3 System Architecture

The proposed Wildfire-Twin system follows a modular, event-driven architecture designed to integrate dynamic fire-related events with large-scale static geospatial infrastructure data. At a high level, the system is organized into four layers: data ingestion, stream processing and enrichment, analytical storage, and interactive visualization. This

separation of concerns makes the pipeline easier to scale, debug, and extend. The repository structure reflects this design directly through dedicated modules for the producer, stream processor, alert sink, storage assets, and dashboard. The initial project vision centered on connecting Kafka-based wildfire events with large static building assets using Spark and Sedona for spatial cross-referencing, and later extending the interface with weather-aware predictive rendering and simulation support.

3.1 Architectural Overview

The end-to-end system can be summarized as follows. Wildfire events, whether originating from live data feeds or user-driven simulations, are published into a Kafka topic named `fire_events`. These events are then consumed by downstream processing components that enrich them with geospatial and environmental context. Static building data are maintained separately in a spatially indexed GeoParquet dataset, while alert results are materialized into a DuckDB-backed live store for fast query access. The dashboard does not consume Kafka directly; instead, it reads already-processed alert data from DuckDB and building context from partitioned GeoParquet files. This decoupled design avoids unnecessary UI stalls and allows the visualization layer to remain responsive even when the upstream stream is active.

A useful way to describe the pipeline is:

Event Sources → Kafka Ingestion → Spark/Sedona Geospatial Processing → DuckDB Alert Sink + GeoParquet Context Store → Streamlit/PyDeck Dashboard

This architecture addresses the three main system requirements of the project: support for high-velocity event ingestion, efficient handling of high-volume static spatial data, and intuitive human-facing visualization of both live and simulated wildfire risk.

3.2 Data Ingestion Layer

The ingestion layer is responsible for generating or receiving wildfire-related events and publishing them into the streaming backbone. Kafka serves as the primary message broker, with producers publishing JSON events to the `fire_events` topic. The baseline event contract includes a timestamp, sensor identifier, latitude, longitude, and temperature fields.

In addition to producer-based ingestion, the system includes a simulation workflow within the dashboard. Users can select coordinates interactively and publish synthetic fire events. The simulation module constructs a Kafka producer, optionally enriches the event with live weather data, and emits a JSON payload containing event time, event ID, sensor ID, latitude, longitude, temperature, fire status, wind speed, wind direction, and humidity.

This dual design supports both simulated and extensible live data sources, aligning with the project vision of integrating real-world wildfire feeds such as NASA FIRMS in future iterations.

3.3 Static Geospatial Data Layer

The static geospatial layer provides infrastructure context for wildfire risk analysis. The primary dataset consists of California-wide building footprints derived from Microsoft Maps. The original GeoJSON dataset was converted into line-delimited JSON and subsequently into Apache Sedona-compatible GeoParquet for efficient distributed processing. The resulting dataset contains over 11.5 million building polygons.

To enrich this dataset semantically, OpenStreetMap point-of-interest (POI) data were integrated. Using Spark and Sedona, a spatial join based on `ST_Contains` was performed between POI points and building polygons. This process

resulted in a dataset of approximately 52,000 essential buildings categorized into infrastructure types such as hospitals, schools, emergency services, and civic facilities.

This enrichment transforms raw geometric data into meaningful infrastructure context, enabling the system to identify and prioritize high-value assets during wildfire events.

3.4 Stream Processing and Geospatial Reasoning Layer

The stream processing layer is responsible for transforming incoming fire events into actionable insights. Apache Spark integrated with Apache Sedona is used to perform distributed geospatial processing.

Each incoming event is converted into a geometric risk region. Initially, this was modeled as a circular buffer, but the design evolved into a directional model influenced by environmental factors such as wind speed and wind direction. This results in a more realistic representation of wildfire spread.

The system then performs spatial intersection between the generated risk region and the enriched building dataset. Each intersecting building produces an alert record, which is forwarded to downstream storage. This layer effectively combines event streaming, environmental context, and spatial analytics into a unified processing workflow.

3.5 Alert Sink and Analytical Storage Layer

To decouple processing from visualization, the system stores alert outputs in a DuckDB-based analytical layer. Instead of directly querying streaming data, the dashboard retrieves alert records from DuckDB using lightweight queries.

This design provides several advantages: reduced coupling between frontend and backend systems, improved query performance, and support for filtering between live and simulated events. DuckDB acts as a low-latency query engine for dynamic data, while GeoParquet remains the primary storage for static spatial context.

3.6 Dashboard and Visualization Layer

The visualization layer is implemented using Streamlit and PyDeck, providing an interactive interface for exploring wildfire risk and infrastructure data.

At the statewide level, the system uses H3-based aggregation to represent building density efficiently. Approximately 200 hexagonal cells are rendered, reducing computational load. At finer zoom levels, the system loads detailed building geometries using H3-based partition filtering.

The dashboard integrates multiple PyDeck layers, including GeoJsonLayer for buildings, ScatterplotLayer for fire events, and PolygonLayer for directional risk zones. Alerted buildings are highlighted dynamically, enabling users to identify at-risk infrastructure visually.

The refresh strategy is designed to balance responsiveness and stability. Alert panels update periodically, while map rendering is triggered manually to avoid unnecessary recomputation.

3.7 Simulation and User Interaction Workflow

The system includes an interactive simulation mode that allows users to generate hypothetical wildfire scenarios. Users select a location, optionally incorporate real-time weather data, and inject a synthetic event into the Kafka pipeline.

This event flows through the same processing architecture as live data, ensuring consistency across the system. The simulation capability enables what-if analysis and supports decision-making scenarios by allowing users to explore potential wildfire impacts under different conditions.

3.8 Architectural Strengths

The architecture demonstrates several key strengths. It is modular, separating ingestion, processing, storage, and visualization into distinct layers. It is scalable, leveraging Kafka for streaming, Spark for distributed processing, and partitioned GeoParquet for efficient data access. It is responsive, using multi-resolution visualization strategies to maintain performance. Finally, it is extensible, allowing integration of additional data sources and more advanced wildfire modeling techniques in future iterations.

4 Methodology

The methodology of the Wildfire-Twin system is designed to integrate real-time event streaming with large-scale geospatial data processing and interactive simulation. The system combines dynamic wildfire events with static infrastructure datasets to enable near real-time risk assessment. The overall methodology can be decomposed into six key components: problem formulation, event modeling, geospatial data preparation, spatial indexing and optimization, wildfire risk modeling, and alert generation with visualization.

4.1 Problem Formulation

The core objective of the system is to determine which infrastructure assets are at risk given an active or simulated wildfire event. This requires combining two fundamentally different data types: dynamic event streams representing wildfire activity and static geospatial datasets representing infrastructure.

The problem is formulated as a continuous spatial query system in which incoming fire events are transformed into geographic risk regions and intersected with a semantically enriched building dataset. The output of this process is a set of infrastructure elements that fall within the predicted risk zone. This formulation allows the system to operate as a real-time decision-support tool rather than a static visualization platform.

4.2 Event Representation and Ingestion

Wildfire events are represented as structured JSON objects containing spatial and environmental attributes. The baseline schema includes event time, latitude, longitude, and temperature, while extended simulation events include additional fields such as wind speed, wind direction, humidity, and fire status.

Events are generated either through producer scripts or through an interactive simulation interface. The simulation workflow allows users to inject synthetic events into the system, optionally enriched with live weather data. All events are published to a Kafka topic, ensuring a unified event stream regardless of source. This standardization enables consistent downstream processing and simplifies integration with future real-world data sources.

4.3 Geospatial Data Preparation and Semantic Enrichment

The static spatial context of the system is derived from large-scale building footprint data and enriched using point-of-interest datasets. The original California building dataset is transformed from GeoJSON into GeoParquet format to support efficient distributed processing.

To assign semantic meaning to building geometries, a spatial join is performed between building polygons and OpenStreetMap POIs using spatial containment operations. This process identifies essential infrastructure such as hospitals, schools, and emergency facilities. The result is a reduced but highly informative dataset of semantically enriched buildings, which serves as the primary reference for risk analysis.

This step is critical because it converts raw geometric data into meaningful entities that can be interpreted within the context of disaster response and infrastructure resilience.

4.4 Spatial Indexing and Data Access Optimization

To ensure scalability and efficient querying, the system employs hierarchical spatial indexing using the H3 hexagonal grid system. The building dataset is partitioned based on H3 indices, allowing selective loading of spatial subsets during queries.

At the statewide level, building data are aggregated into coarse H3 cells to reduce rendering complexity. At finer geographic scales, the system dynamically retrieves only the relevant partitions using H3 neighborhood expansion and filtering. This approach avoids full dataset scans and significantly improves performance during interactive exploration.

Caching mechanisms are also employed to minimize repeated data loading, further enhancing responsiveness.

4.5 Wildfire Risk Modeling

The system models wildfire impact using a geometric approximation of fire spread. Initially, risk regions were defined using circular buffers around fire points. This approach was later extended to incorporate environmental factors, resulting in a directional risk model influenced by wind speed and wind direction.

The directional model generates a cone-shaped region representing probable fire spread. The extent of this region is scaled based on wind speed, while its orientation is determined by wind direction. Additional environmental parameters such as temperature and humidity can further influence the effective risk zone.

This approach provides a balance between computational efficiency and practical realism, enabling near real-time risk estimation without requiring complex physical simulation models.

4.6 Alert Generation and Visualization Integration

Once a risk region is constructed, the system performs spatial intersection between the risk geometry and the semantically enriched building dataset. Each intersecting building is identified as being at risk and converted into an alert record.

These alert records are stored in a DuckDB-based analytical layer, enabling fast query access for the dashboard. The visualization layer then retrieves these alerts and integrates them with geospatial context for rendering.

The dashboard presents results using multiple visualization layers, including building geometries, fire event markers, and predicted risk regions. At-risk buildings are highlighted dynamically, allowing users to quickly identify critical infrastructure under threat.

Additionally, the system supports simulation-driven analysis, where user-generated events follow the same pipeline as real data. This integration ensures consistency across monitoring and exploratory workflows.

5 Implementation Details

This section describes the practical realization of the Wildfire-Twin system, focusing on system components, technology choices, data processing pipelines, and optimization strategies used to ensure scalability and responsiveness.

5.1 Technology Stack and System Setup

The system is implemented using a combination of streaming, distributed processing, analytical storage, and visualization technologies. Apache Kafka is used as the message broker for ingesting wildfire events, deployed using a Dockerized setup to simplify configuration and reproducibility. Producers publish JSON-formatted events to a topic named `fire_events`.

Apache Spark, integrated with Apache Sedona, is used for distributed geospatial processing. The processing environment supports spatial SQL operations such as geometric buffering and intersection. Static datasets are stored in GeoParquet format, enabling efficient columnar access and compatibility with Spark-based workflows.

DuckDB is used as a lightweight analytical database for storing alert outputs. Its in-process execution model enables low-latency queries directly from the dashboard backend. The frontend is implemented using Streamlit, with PyDeck used for WebGL-based geospatial rendering. The H3 spatial indexing library is used for hierarchical partitioning and multi-resolution visualization.

5.2 Data Pipeline Implementation

The data pipeline begins with event generation through either producer scripts or the dashboard simulation interface. Each event is serialized as a JSON object and published to Kafka. The event schema includes spatial attributes (latitude and longitude), temporal attributes (event time), and environmental parameters such as temperature, wind speed, wind direction, and humidity.

On the processing side, events are consumed and transformed into geospatial objects. Each event is converted into a risk geometry, initially modeled as a circular buffer and later extended into a directional polygon based on wind parameters. Apache Sedona functions are used to perform geometric operations such as buffering and spatial intersection.

The system then performs a spatial join between the generated risk region and the semantically enriched building dataset. Each intersecting building is converted into an alert record, which includes metadata such as building type, location, and associated event identifiers. These alerts are then written to the DuckDB analytical layer.

5.3 Geospatial Data Engineering

The static geospatial pipeline involves multiple transformation stages. The original building dataset, provided in GeoJSON format, is first converted into line-delimited JSON to improve ingestion reliability. It is then transformed into GeoParquet format using Apache Sedona, enabling efficient distributed access and columnar storage.

To incorporate semantic information, OpenStreetMap POI data are spatially joined with building polygons using `ST_Contains`. This results in a filtered dataset of approximately 80,000 essential buildings categorized into infrastructure types such as healthcare, education, and emergency services.

The resulting dataset is partitioned using H3 indices at resolution 7. Each partition corresponds to a spatial region, enabling selective loading of relevant subsets during query execution. This partitioning strategy significantly reduces I/O overhead during interactive visualization.

5.4 Storage and Query Layer

The system uses a hybrid storage strategy to separate static and dynamic data. Static building data are stored in partitioned GeoParquet files, serving as the authoritative source for infrastructure geometry. Dynamic alert data are stored in DuckDB, providing fast query access for the dashboard.

The DuckDB schema is designed to support efficient retrieval of recent alerts and aggregation queries. Helper functions in the backend application handle loading of alerts and computing summary statistics such as alert counts. This separation between static and dynamic storage allows the system to maintain performance while handling both large datasets and real-time updates.

5.5 Optimization Strategies

Several optimization techniques are implemented to ensure system scalability and responsiveness. H3-based partition pruning is used to restrict GeoParquet reads to only the relevant spatial regions for a given query. This avoids full dataset scans and reduces data loading time.

Caching mechanisms are employed within the Streamlit backend using `@st.cache_data`, reducing redundant I/O operations for frequently accessed data. At the visualization level, multi-resolution rendering is used to switch between aggregated H3 representations and detailed building geometries based on zoom level.

The architecture also decouples the dashboard from the streaming pipeline by introducing DuckDB as an intermediate layer. This prevents UI blocking and ensures that visualization remains responsive even under continuous event ingestion.

5.6 System Integration and Execution Workflow

The complete system operates as a loosely coupled pipeline connecting multiple components. Kafka handles event ingestion, Spark and Sedona perform geospatial processing, DuckDB stores alert outputs, and Streamlit serves the interactive dashboard.

Execution is managed through modular scripts that initialize different components of the system. The ingestion layer can be started independently, followed by the processing layer and finally the dashboard application. This modular design simplifies debugging and allows individual components to be tested in isolation.

During operation, events flow from Kafka through the processing pipeline into the alert store, and are then retrieved by the dashboard for visualization. This end-to-end workflow enables near real-time monitoring and interactive exploration of wildfire risk across large-scale geospatial datasets.

5.7 Implementation Challenges

Several practical challenges were encountered during system development. Handling the large GeoJSON building dataset required intermediate format conversion to ensure compatibility with distributed processing frameworks. Spatial joins at scale required careful optimization to avoid performance bottlenecks.

Rendering large numbers of geospatial objects in the browser posed additional challenges, which were addressed through H3-based aggregation and selective loading strategies. Ensuring low-latency interaction between backend processing and frontend visualization required decoupling components and introducing efficient caching mechanisms.

Despite these challenges, the final system achieves a balance between scalability, responsiveness, and practical usability, demonstrating the feasibility of integrating streaming and geospatial analytics in a unified digital twin architecture.

6 Evaluation

This section evaluates the Wildfire-Twin system in terms of dataset scale, system performance, responsiveness, and overall scalability. Given the system's focus on real-time geospatial analytics, evaluation emphasizes both computational efficiency and practical usability.

6.1 Dataset Characteristics

The system operates on a large-scale geospatial dataset derived from Microsoft's California building footprint collection. The dataset contains approximately 11.5 million building polygons, representing one of the primary scalability challenges in the system.

After semantic enrichment using OpenStreetMap point-of-interest (POI) data, approximately 80,000 buildings were identified as essential infrastructure. These include hospitals, schools, emergency facilities, and other critical assets. This reduction from raw geometry to semantically meaningful infrastructure enables focused and efficient risk analysis.

6.2 System Performance and Latency

The system is designed to support near real-time processing of wildfire events. While precise end-to-end latency from Kafka ingestion to dashboard visualization is still under measurement, the architecture ensures low-latency performance through streaming ingestion, distributed processing, and lightweight analytical querying.

At the visualization level, the system demonstrates strong responsiveness. Loading and rendering a city-scale view, including building geometries and alert overlays, takes at most 5 seconds under typical conditions. This includes data retrieval from GeoParquet, alert queries from DuckDB, and rendering through PyDeck.

The decoupling of the dashboard from the streaming layer ensures that visualization performance is not degraded by continuous event ingestion, allowing stable and responsive user interaction.

6.3 Scalability and Optimization

To handle large-scale geospatial data efficiently, the system employs multiple optimization strategies. H3-based spatial indexing is used to partition the building dataset, enabling selective loading of relevant spatial regions. At the statewide level, the dataset is aggregated into approximately 270 H3 cells, significantly reducing rendering complexity while preserving spatial coverage.

At finer resolutions, H3-based partition pruning ensures that only relevant GeoParquet partitions are accessed, avoiding full dataset scans. This approach reduces I/O overhead and improves query performance during interactive exploration.

Caching mechanisms within the Streamlit backend further improve performance by minimizing repeated data loading operations. Together, these techniques allow the system to scale effectively across both data size and user interaction complexity.

6.4 Functional Validation

The system was validated through simulation-driven experiments. Synthetic wildfire events were generated using the dashboard interface, with parameters such as location, temperature, wind speed, and wind direction.

For each simulated event, the system successfully generated risk regions, performed spatial intersection with the enriched building dataset, and produced alert records for affected infrastructure. These alerts were correctly reflected in the dashboard through highlighted buildings and updated alert panels.

The directional wildfire model produced intuitive and interpretable risk regions aligned with wind conditions, demonstrating the effectiveness of incorporating environmental parameters into the risk modeling process.

6.5 System Limitations

Despite its strengths, the system has several limitations. First, the current implementation relies primarily on simulated events, with full integration of live wildfire data sources such as NASA FIRMS left for future work. Second, the wildfire spread model is based on geometric approximations and does not incorporate complex physical factors such as terrain, vegetation type, or fuel load.

Additionally, while the system supports near real-time interaction, full end-to-end latency measurements under high-throughput conditions have not yet been extensively benchmarked. These limitations highlight opportunities for further refinement and extension.

6.6 Summary of Evaluation

Overall, the evaluation demonstrates that the Wildfire-Twin system can process and visualize large-scale geospatial datasets while maintaining interactive performance. The combination of streaming architecture, spatial indexing, and efficient data storage enables the system to operate effectively as a real-time digital twin for wildfire risk analysis.

7 Conclusion

The Wildfire-Twin system demonstrates how real-time streaming architectures can be effectively combined with large-scale geospatial analytics to build a responsive and scalable digital twin for disaster monitoring. By integrating Kafka-based event ingestion, Spark and Apache Sedona for distributed spatial processing, and DuckDB for low-latency analytical querying, the system successfully bridges dynamic wildfire events with static infrastructure data.

A key strength of the system lies in its ability to operate across multiple spatial scales. The use of H3-based indexing and partitioned GeoParquet storage enables efficient handling of over 11.5 million building geometries while maintaining interactive performance in the dashboard. At the same time, the incorporation of directional wildfire risk modeling introduces a more realistic approximation of fire spread compared to naive buffer-based approaches.

The system also extends beyond passive monitoring by supporting simulation-driven workflows. Users can inject hypothetical fire events enriched with environmental context and observe their impact on critical infrastructure in near real time. This capability aligns closely with the concept of a digital twin, enabling both situational awareness and exploratory analysis.

Overall, the project validates that combining streaming systems, distributed geospatial processing, and interactive visualization can produce a practical and extensible platform for wildfire resilience analysis. The architecture provides a strong foundation for future integration of real-world data sources and more advanced predictive models.

8 Future Work

While the current system establishes a functional and scalable digital twin, several directions exist for meaningful extension and improvement.

First, integration of real-world wildfire data sources such as NASA FIRMS would enable the system to transition from simulation-heavy workflows to fully live monitoring. Continuous ingestion of satellite-based fire detections would significantly enhance the system's practical applicability.

Second, the wildfire risk model can be improved by incorporating more sophisticated environmental dynamics. Current directional buffering based on wind parameters can be extended to include terrain, vegetation type, fuel load,

and historical fire spread patterns. Machine learning or physics-informed models could further improve predictive accuracy.

Third, the system can benefit from deeper temporal analytics. Currently, alerts are generated based on instantaneous events, but extending the pipeline to support temporal aggregation, trend analysis, and multi-event correlation would provide richer insights into wildfire progression over time.

Fourth, scalability improvements can be explored at both the processing and visualization layers. While H3 indexing and partition pruning already optimize performance, future work could include distributed query engines, adaptive caching strategies, and progressive rendering techniques for even larger datasets.

Finally, the user interface can be expanded to support decision-making workflows. Features such as evacuation planning, infrastructure prioritization, and alert severity scoring would transform the system from a monitoring dashboard into a comprehensive decision-support tool.

Together, these extensions would move the system closer to a fully operational, real-world wildfire resilience platform capable of supporting emergency response and long-term planning.

Author Contributions

Prince Savsaviya led the development of the data ingestion layer, including Kafka-based event production and schema design for wildfire events. He also contributed to integrating simulation-generated events into the streaming pipeline.

Viswanadh R. Challa was responsible for the stream processing and geospatial analytics layer, implementing the Spark and Apache Sedona pipeline for spatial joins, risk region construction, and infrastructure intersection logic.

Ali Rezayi Nejad focused on the dashboard and visualization layer, including the design and implementation of the Streamlit interface, PyDeck-based geospatial rendering, and integration of alert data with interactive map components.

Anish Ashok Kale worked on the data preparation and storage components, including transformation of raw geospatial datasets into GeoParquet, semantic enrichment using OpenStreetMap data, and integration of DuckDB as the analytical alert store.

All authors contributed to system integration, debugging, and end-to-end testing of the pipeline.

References